

— TRAINING-FREE · NO TASK HEAD · OPEN VOCABULARY

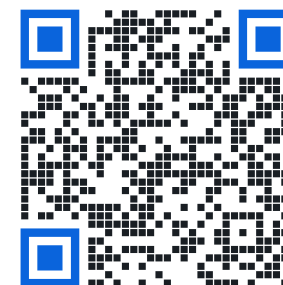
# Test-time compute of jina-embeddings-v5-omni for image tagging

**Han Xiao**

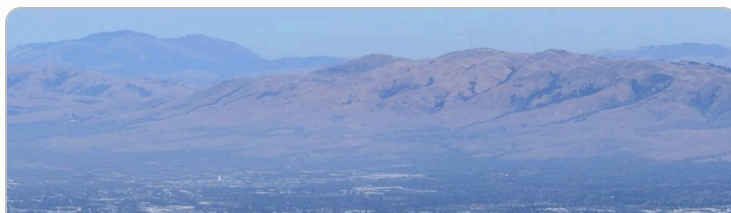
VP of AI, Elastic

[@hxiao](#) · [in/hxiao87](#)

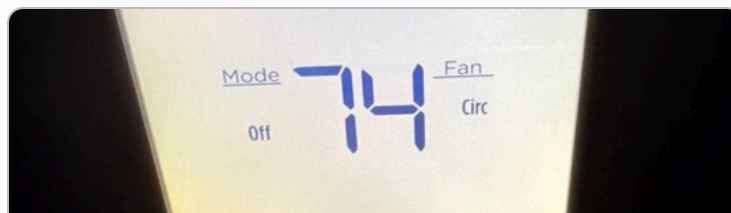
THIS DECK



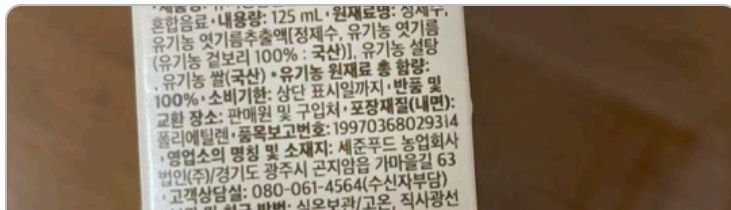
# Training-free, open-vocabulary tagging from a frozen embedding model.



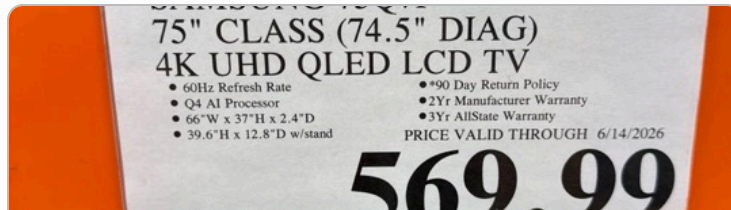
mountain · phoenix · sky · rooftop · location



clock · thermostat · dial · dst · wifi



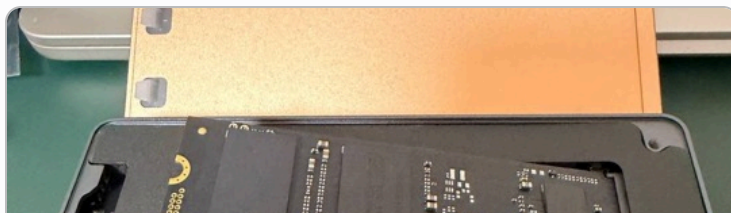
contiene · liters · dosage · thirst · consumo



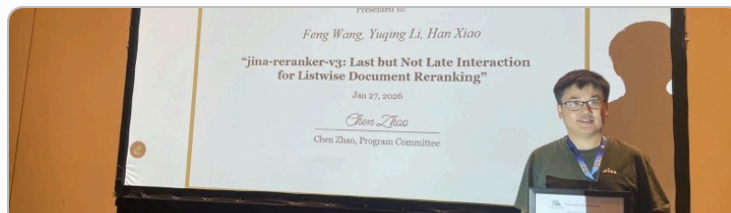
harga · lcd · samsung · refurbished · warrants



fiyat · pagamento · brasile · gratuiti · utils



tablet · motherboard · wearable · celular · datastore



award · forwarding · speaking · defendant · workshop



gift · toy · celebration · balloons · cheerful

## Problem formulation.

- | GIVEN one embedding model, **jina-embeddings-v5-omni-nano** (~1B, image and text in one 768-d space), and one image.
- | OUTPUT words for **every** object in the image: multi-label, open vocabulary.
- | FROZEN all weights. The model is a retrieval encoder; it has no tagging head, no classifier, and was never trained for this task.
- | ALLOWED inference-time computation over the model's own outputs, and nothing else.

NO TRAINING

NO SECOND MODEL

NO WORDNET / DICTIONARY

NO REGEX / POS TAGGER

Whatever tagging ability appears must come from [test-time compute of the embedding model](#).

## Where new information comes from.

A frozen encoder's inference-time budget takes roughly three forms:

### A DEEPER PASS

Read more of the one pass you already ran: every patch vector, not just the pooled one. The retrieval world's **late interaction**.

**new information: reclaimed from inside the pass**

### B MORE PASSES

Run the frozen encoder again on new views: split a document, crop an image, re-encode.

**new information: acquired from the input**

### C BOOTSTRAP

Transform the vectors you already have: whitening, propagation, optimal transport. Where most of the training-free literature lives.

**new information: none**

Same frozen encoder in all three; they differ only in **what enters the computation**.

## Two research questions.

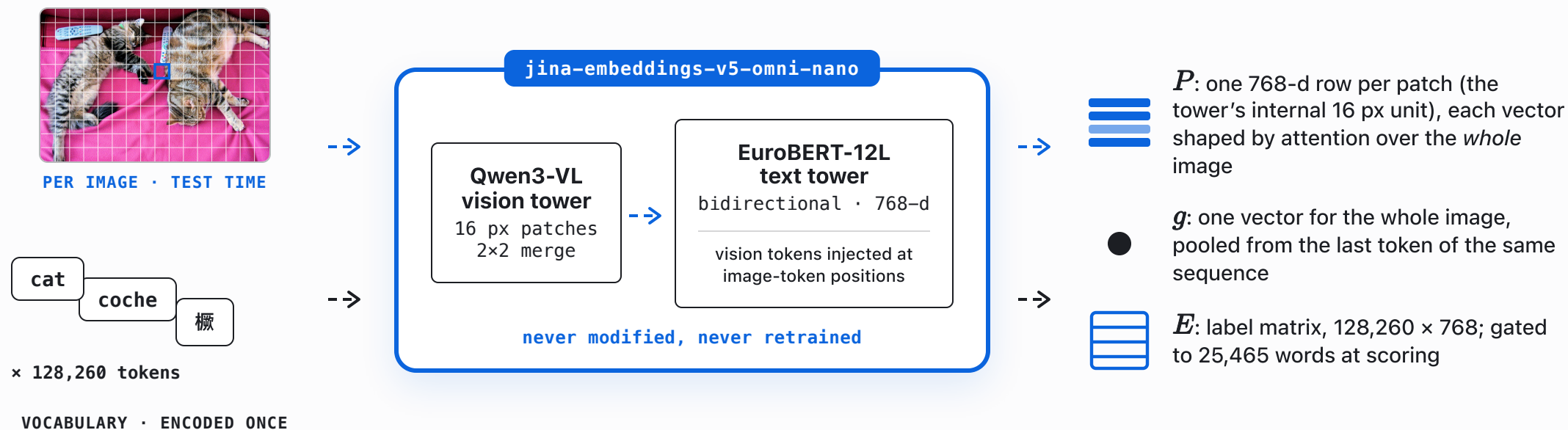
### **RQ1** Can a frozen embedding model become an image tagger through test-time compute alone?

Prior tagging work pays with training or resources: Tag2Text / RAM / RAM++ train over ~6,400 curated tags; TagCLIP (AAAI'24) and PIAA ("[CLS] is not enough") are training-free but assume a given class list, on two-tower CLIP plus WordNet or POS tools.

### **RQ2** If it can: which test-time compute scales its accuracy, bootstrapping or new information?

The 2024–26 literature bets on family C: whitening (PIAA), label propagation (ZLaP), optimal transport (OTTER), Bayesian adaptation (BCA).

## Image and text embedding space.

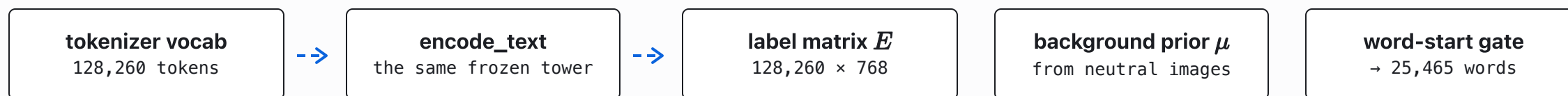


Both paths end in the same space, so  $\langle p, e_\ell \rangle$  is well-defined: **any patch of any image, scored against any word the model knows.**

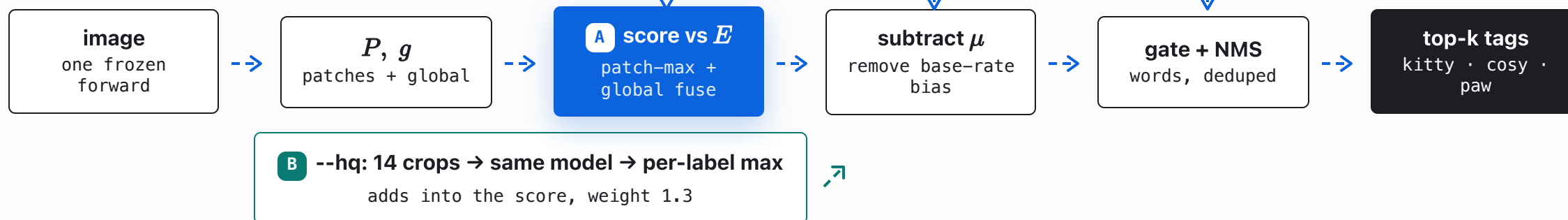
## — THE PIPELINE

# The complete tagger workflow as test-time compute.

### OFFLINE



### PER IMAGE



Every box is the frozen model or plain arithmetic: **A** reads more of one pass; **B** runs new passes on new pixels.

## The label set is the tokenizer's own vocabulary.

No curated tag list. Take all **128,260** tokens and encode each one as text into a label matrix

$$E = [e_\ell]_{\ell \in V}, \quad e_\ell = \text{encode\_text}(\ell) \in \mathbb{R}^{768}, \quad |V| = 128,260$$

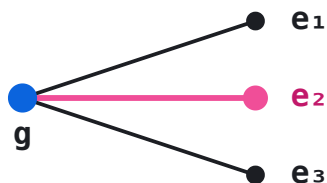
One matmul now scores any image against the entire vocabulary; fragments are filtered later by a word-start gate (step 4).

```
wrong: image_vec · embed_tokens.weight  
       → Tutor, avatar, PyTuple (garbage)  
right: image_vec · encode_text(token)  
       → kitty, cosy, plush, paw (aligned)
```

**Why encode\_text, not the embedding table?** The model has `tie_word_embeddings = false`: the input-embedding table lives in a different space than the pooled output. You must encode each token *as a text string* to put labels in the image's space.

## Score every patch.

GLOBAL POOLED

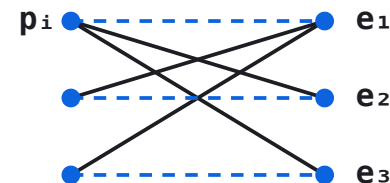


$$s_\ell = \langle g, e_\ell \rangle$$

one vector dominated by the most salient object

mAP 0.264

PER-LABEL MAX OVER PATCHES



$$s_\ell = 0.7 \max_{p \in P} \langle p, e_\ell \rangle + 0.3 \langle g, e_\ell \rangle$$

every label takes its max over all patches: late interaction (ColBERT), with patches as the token vectors

mAP 0.635

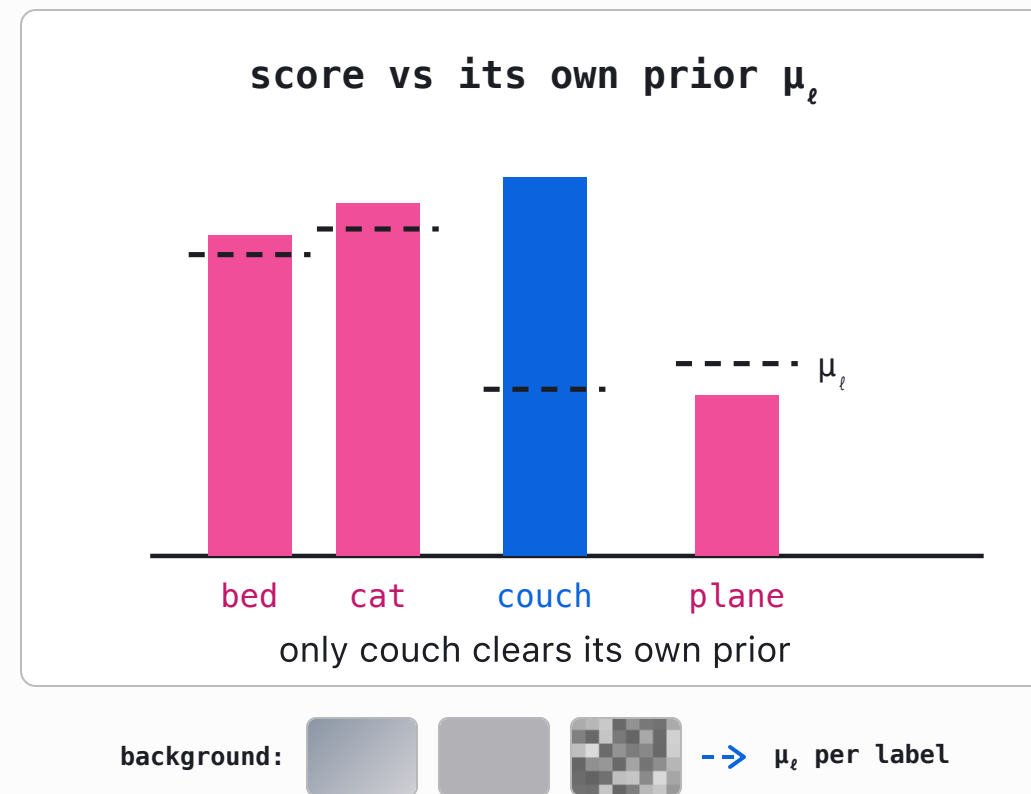
**A** PIAA's "[CLS] is not enough", confirmed: a small object fires on its own patch and survives the max. **+0.37 mAP**, from rows the pass already produced.

## Subtract the per-label prior.

Raw cosine has a base-rate bias: generic words ("bed", "cat") score high on almost any image because they sit near the image modality. Estimate a per-label prior once from neutral background images, then subtract it:

$$\tilde{s}_\ell = s_\ell - \mu_\ell, \quad \mu_\ell = \mathbb{E}_{x \sim \text{background}} [s_\ell(x)]$$

No calibration set, no labels, one offline pass.



## The tokenizer already knows what a word is.

**Word-start gate.** Byte-BPE tokenizers encode a leading space as the glyph `Ġ`; a token beginning with it starts a new word: `Ġcat` is a whole word, `GetComponent` is a fragment. That one built-in signal is the gate; no external dictionary.

**Embedding-NMS.** Synonyms and multilingual variants ( `кот` / `Cat` / `кот` / `cat`) collapse into one, using cosine in the model's own space. Walk labels by score; keep  $\ell$  only if  $\max_{k \in \text{kept}} \langle e_\ell, e_k \rangle < 0.6$ . Again: no lookup table, just the geometry.

### raw vocabulary

128,260 tokens

`Ġkitty` · `Ġcat` · `GetComponent` · `_cpp` · `猫` · `quisites`

↓ word-start gate: keep tokens with the leading-space mark `Ġ`

### whole words

25,465

`kitty` · `cat` · `kitten` · `chatte` · `kitt` · `cats` · `kittens` · `cosy` · ...

↓ embedding-NMS: walk down by score; drop any word with cosine  $\geq 0.6$  to one already kept

### distinct concepts

top-k

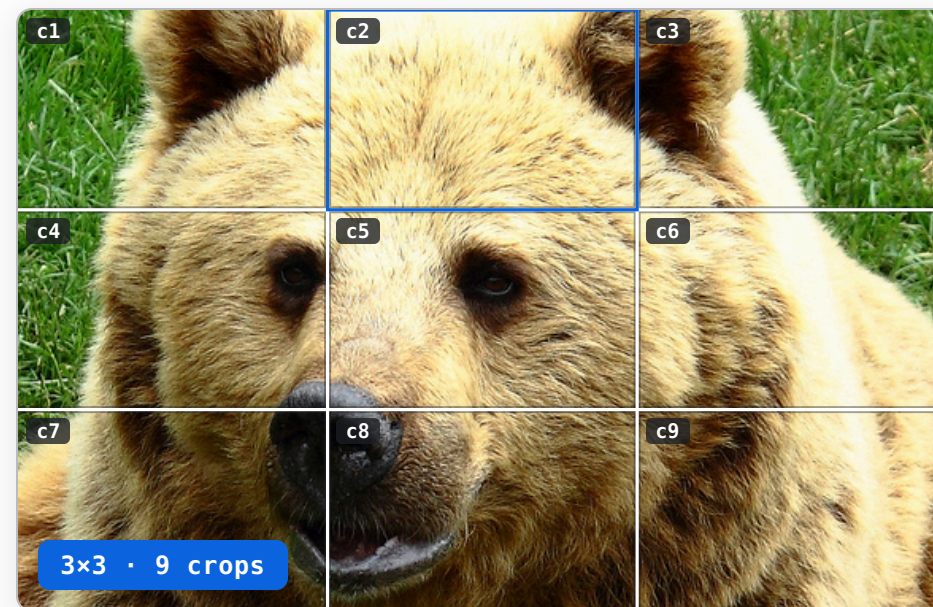
`kitty` · `cat` · `kitten` · `chatte` · `cats` · `cosy` · `plush` · `crib`

## Multi-crop re-encoding.

**Class-Wise Reranking (CWR)**, the crop-and-rescore idea from TagCLIP (AAAI'24), extended to a full-coverage grid of 14 crops (overlaid right) through the same frozen model. A crop is a *new input image*, itself re-patchified by the tower; a small object fills whichever crop contains it, so weak signal becomes strong.

$$S_\ell = \tilde{s}_\ell + 1.3 \left( \max_{c=1..14} s_{c,\ell} - \mu_\ell \right), \quad s_{c,\ell} = \max \left( \max_{p \in P_c} \langle p, e_\ell \rangle, \langle g_c, e_\ell \rangle \right)$$

**B** Per-label **max**, not averaging: **averaging hurt**. The outlier crop is the evidence.



fast: **wolf**, **roar**, **muzzle** → --hq: **fur**, **bear**, **muzzle**  
 multi-crop re-encoding corrects the single-pass misreading

## The entire tagger is one scoring function.

the same score , applied to each crop as a fresh image


$$S(\ell) = \underbrace{0.7 \left( \max_{p \in P} \langle p, e_\ell \rangle - \mu_\ell \right)}_{\text{patch evidence}} + \underbrace{0.3 \left( \langle g, e_\ell \rangle - \mu_\ell^g \right)}_{\text{global context}} + \underbrace{1.3 \left( \max_{c=1..14} s_\ell(\text{crop}_c) - \mu_\ell \right)}_{\text{multi-crop re-encode}}$$

**A Patch evidence:** algebra over rows the forward pass already produced. Free. +0.37 mAP.

**A Global context:** the pooled vector, de-biased by its own background prior. Free.

**B Multi-crop:** 14 fresh forward passes on new pixels. 14x the cost. +0.075 mAP.

then  $\ell \in \text{word-gated vocabulary} \rightarrow \text{embedding-NMS } (\tau = 0.6) \rightarrow \text{top-}k$

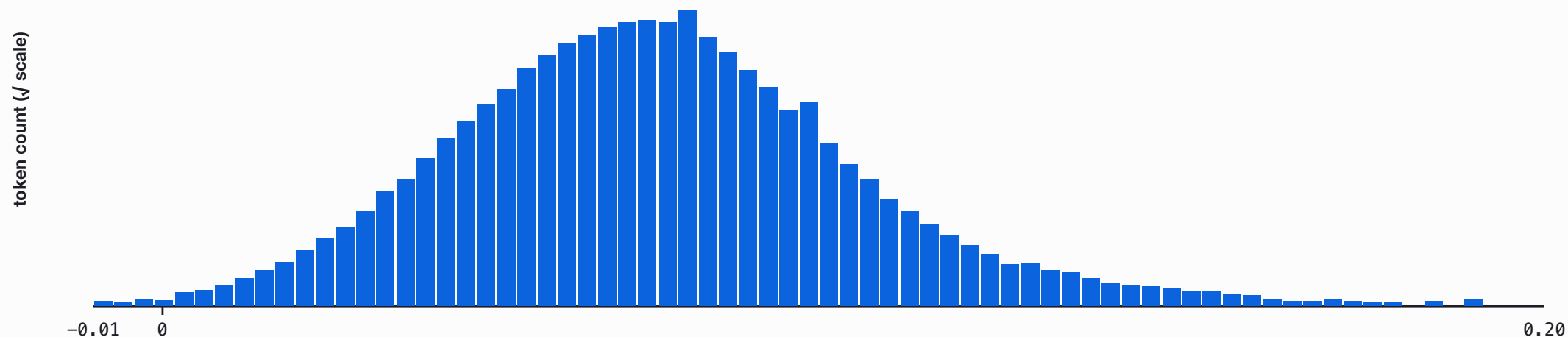
No head, no logits, no learned threshold: **three fixed weights, two background priors, and max operations over frozen-model outputs.**

— THE EQUATION, VISUALIZED

# The pipeline sharpens a distribution.

1 · raw fused score

all 128,260 tokens · 0.7 patch-max + 0.3 global



a smooth bell: every token is somewhat close to every image

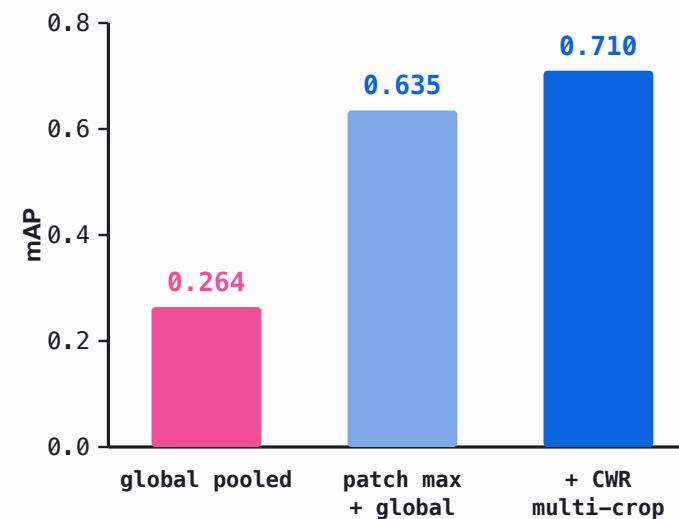
Real pipeline output, one image, all 128,260 tokens. Tagging is distribution sharpening.

## RESULTS

# Results on COCO-150.

| METHOD             | P@1   | P@3   | R@5   | MAP   |
|--------------------|-------|-------|-------|-------|
| global pooled      | 0.433 | 0.289 | 0.449 | 0.264 |
| patch max + global | 0.753 | 0.427 | 0.631 | 0.635 |
| softpool           | 0.773 | 0.449 | 0.649 | 0.608 |
| + CWR multi-crop   | 0.813 | 0.476 | 0.680 | 0.710 |

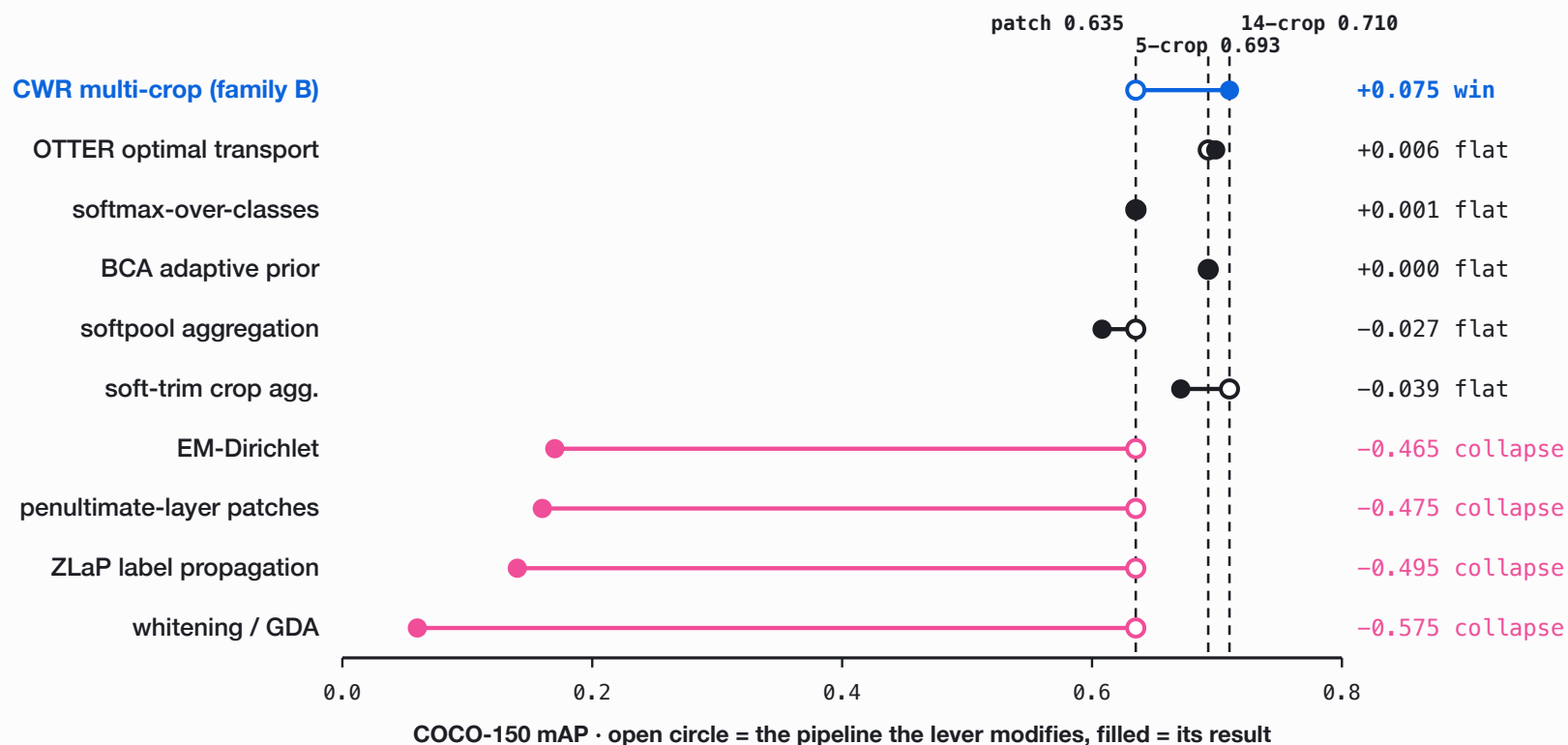
150 COCO-val images, real multi-label ground truth, avg 2.93 labels/image. softpool trades mAP for top-k precision; the ladder tracks the max-pool path.



mAP: global → patch → +CWR

# Does bootstrapping scale accuracy at test time?

Take one: **OTTER** re-balances the per-label scores toward a target class distribution by optimal transport — a pure re-arrangement of vectors it already has. It is applied on top of the 5-crop CWR pipeline, so its baseline is **0.693**, not the 0.635 patch baseline (open circle = the pipeline each lever modifies, filled = its result). Result: +0.006, noise.



## Only new information moves accuracy.

Bootstrap methods were designed for weakly calibrated, single-label, two-tower CLIP. This encoder's space is already aligned, calibrated, and multi-label: **there is nothing left to recover.**

### BOOTSTRAP (FAMILY C)

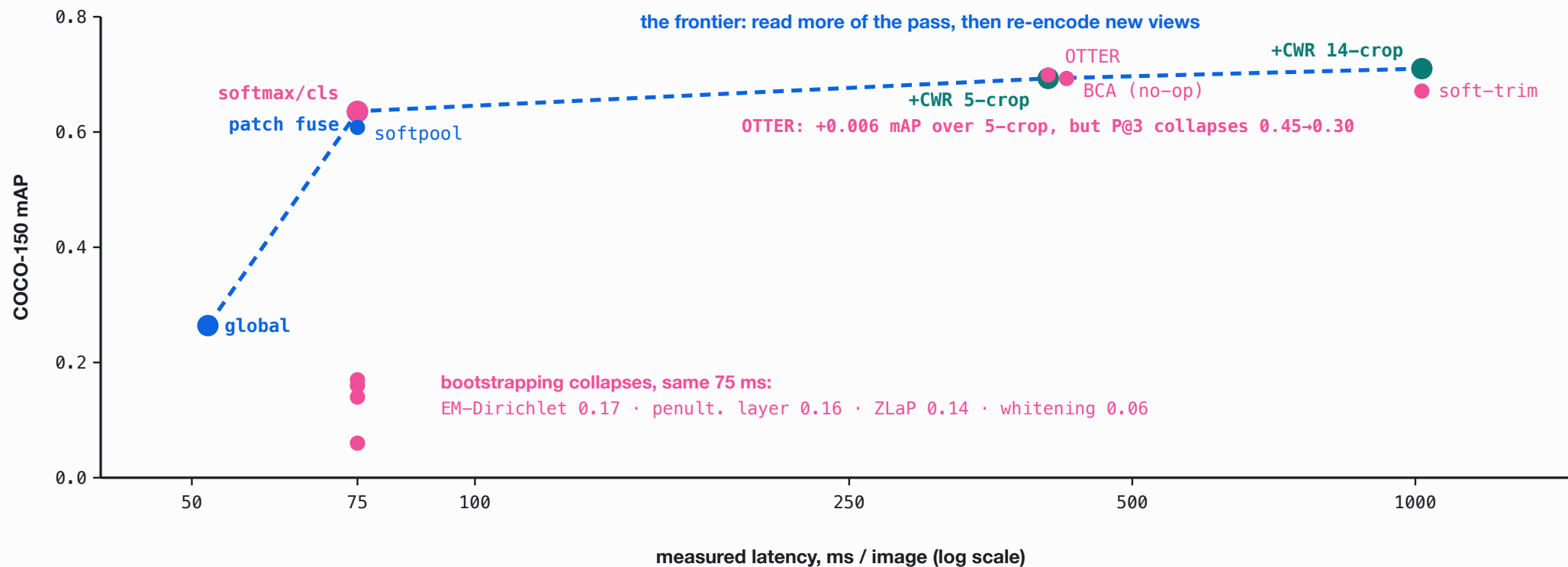
whitening 0.06 · ZLaP 0.14 · EM-Dirichlet 0.17 · OTTER +0.006  
over its own base. None improves the pipeline it modifies.

### FEEDING NEW INFORMATION

CWR crops → mAP 0.710, P@1 0.813. The only intervention  
that improves results.

The ceiling is the 1B model itself. Real test-time compute here means  
**giving the model more to look at**, not re-arranging what it already saw.

## Accuracy versus test-time compute.



The frontier is traced by reading more of the pass and re-encoding new views;  
bootstrapping sits at the same cost, at or below it (<0.1 ms/img, measured).

## Extend to n-grams.



the same image, the same frozen model, one extra scoring pass

### THE SAME FOUR TAGS, N = 1, 2, 3

|     |                  |                     |                     |                  |
|-----|------------------|---------------------|---------------------|------------------|
| N=1 | kitty            | cosy                | plush               | crib             |
| N=2 | couch kitty      | black cosy          | grey plush          | grey crib        |
| N=3 | grey couch kitty | sleeping black cosy | grey sleeping plush | sleeps grey crib |

### how the modifier is chosen · noun = kitty

no word-class constraint

pool the patches where **kitty** fires → rank all 25,465 gated words on that region alone:

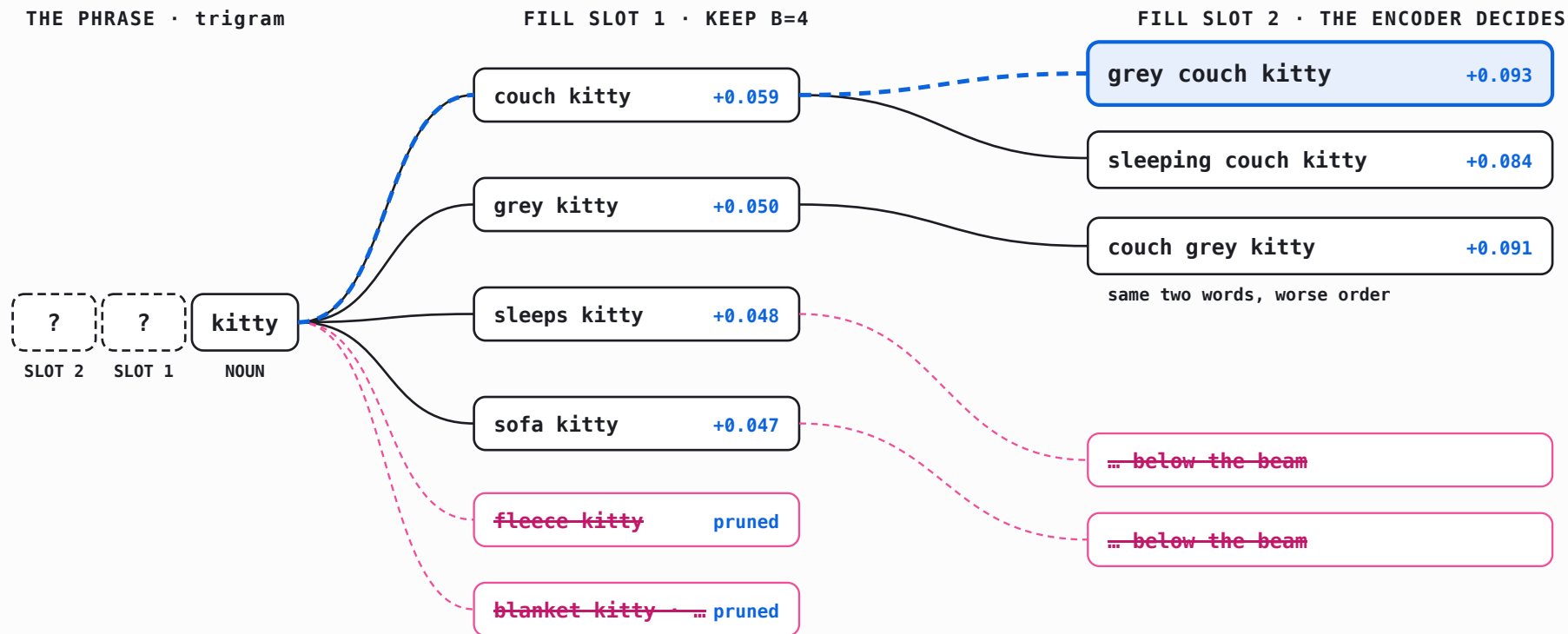
~~cat~~ · **kitty** · ~~kitten~~ (cos ≥ 0.55 to the noun: suppressed) · **couch** ✓ · fleece · blanket · grey

→ greedy picks off one ranking · --beam lets the encoder score whole phrases against the region: it promotes “grey couch kitty” and prefers attribute-first order, with no grammar anywhere

## Beam search over n-gram slots.

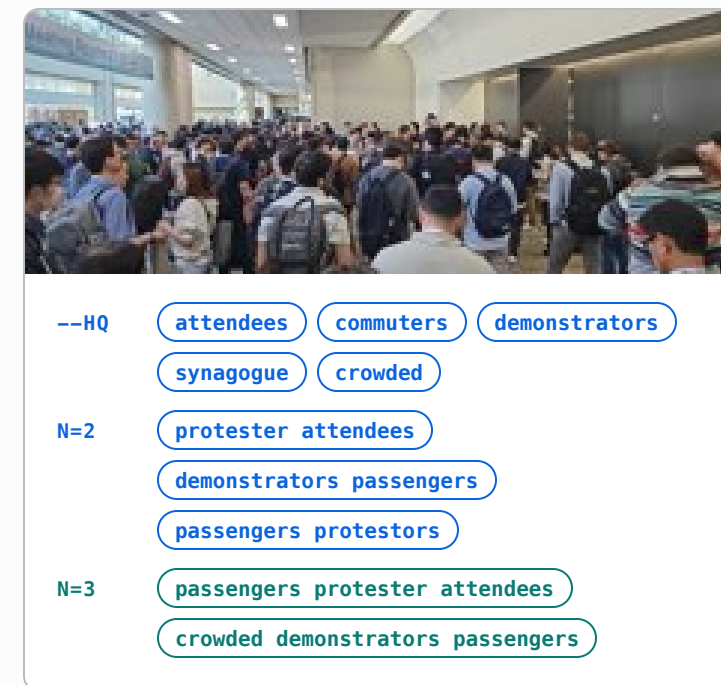
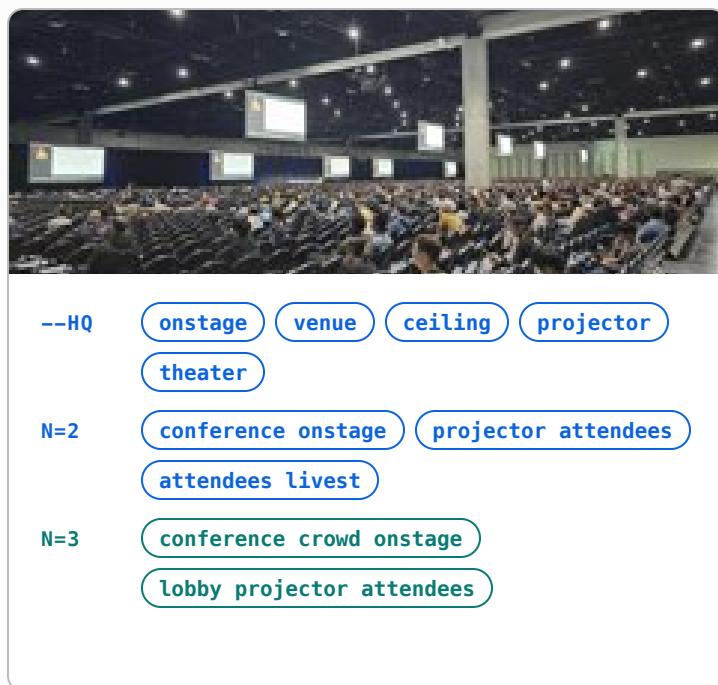
Every node is a real phrase, embedded by the frozen model and scored against the noun's patch region:

$s = \langle E(\text{phrase}), \text{region} \rangle - \langle E(\text{kitty}), \text{region} \rangle$ . Beam width 4.



The winning path re-orders its own words: "grey couch kitty" beats "couch grey kitty". Word order, resolved by an embedding model with no grammar.

## Out-of-distribution photos.



14-crop re-encoding and beam-searched n-grams,  
on unstaged photos. **All outputs verbatim, unfiltered.**

## Versus the recent tagging literature.

| LINE OF WORK                    | THEIR SETUP                                                 | OUR DELTA                                                      |
|---------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|
| Tag2Text / RAM / RAM++          | trained tagging models, curated ~6.4k tag list              | training-free, label space = tokenizer vocab                   |
| TagCLIP                         | CLIP two-tower, penultimate layer, DMAR+CWR                 | single omni model, last layer is the output space              |
| PIAA                            | patch-level scoring ("[CLS] is not enough") + GDA whitening | patch thesis confirmed (+0.37 mAP); whitening collapses (0.06) |
| recent calibration (OTTER, BCA) | calibration set or target prior to de-bias                  | one offline background prior, zero calibration                 |

**RQ1:** a frozen embedding model does become a tagger, training-free. **RQ2:** only compute that adds information scales it. Two questions, answered.

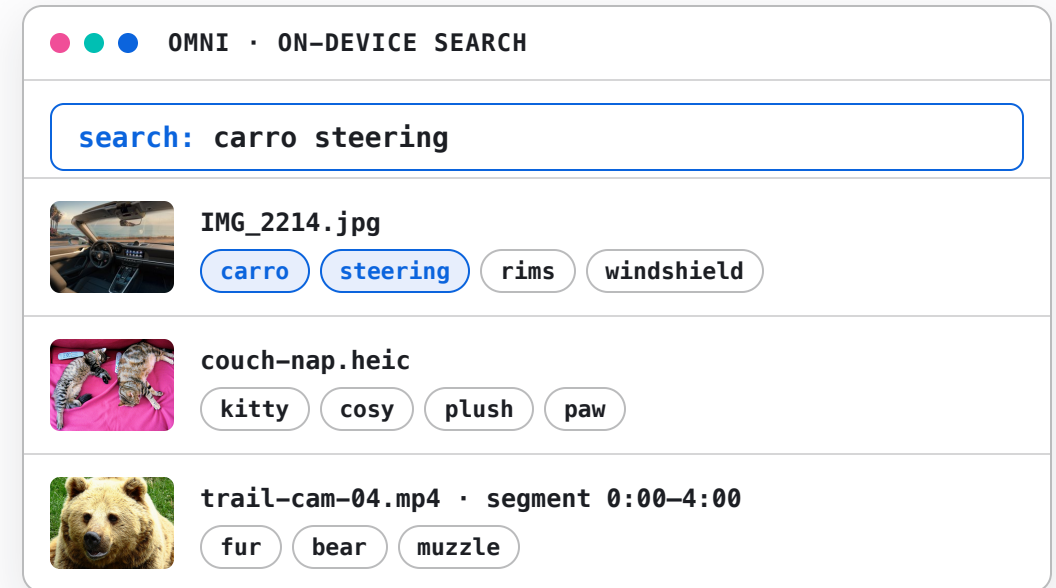


## DEPLOYMENT

# On-device deployment in Omni.

The Python study became a feature of **Omni**, a native on-device semantic-search app (Swift + MLX, same frozen model). The port reuses the architecture directly:

- | SAME FORWARD PASS      the patch rows already exist for the file's *embedding*; tagging adds one matmul.
- | ~0.6 MS / IMAGE      about 4% overhead on the embed step. Tagging is effectively free at index time.
- | TAGS = SNIPPETS      tags land in the search snippet: media becomes *keyword*-searchable.
- | SELF-CALIBRATING      the background prior  $\mu$  is estimated on-device from the first 64 images it sees, then frozen.



tag chips as they appear in results; the third row is a video segment

# Images, video, and scanned documents.



## IMAGE

Tagged at **index time**, inside the embedding forward. Patch-max + global, prior-centered, gated, NMS-deduped.



## HQ IMAGE

A background **re-tag pass** adds the production CWR variant: 5 crops, per-label max.



## VIDEO

32 frames per **240-second segment**, one sequence through the tower: patch-max pools over **space and time**.



## SCANNED PDF

Pages render to images, same path: **per-page tags** ("invoice, table, signature") as the snippet.

## A capability the model was never trained for, realized at inference.

Two talks, one thesis. A frozen encoder holds more capability than its training objective exposes, and you unlock it with **test-time compute**, not more parameters.

### RETRIEVAL TALK

recombine the geometry for more **relevance** on the task it was trained for.

### THIS TALK

recombine the geometry for an **emergent new task** it was never trained for: tagging.

Test-time compute scales only when it **supplies the model with new information**. Bootstrapping an already-good representation yields nothing.

# Scaling test-time compute of an embedding model **unlocks tasks beyond retrieval.**

**Han Xiao**

VP of AI, Elastic

[@hxiao](#) · [in/hxiao87](#)

GITHUB REPO

